Verifying the Blueprint

High Level Design and Low Level Design — the two drawings every system is built from.

UNIVERSITY QUESTION · MU — MAY 18

"Explain verification in high level and low level design."

Continues directly from §1.10 — verification is static, done before any code runs.

· THE CHEAPEST PLACE TO CATCH A FAULT

No builder pours a foundation without checking the architect's drawings first.

A design is a document — so it is verified, not validated: examined statically through reviews, walkthroughs, and inspections, with no code executed. A flaw caught in the design is a corrected paragraph. The same flaw caught after coding is a rebuilt system. That is why design verification is among the highest-return activities in the whole life cycle.

High Level Design — The Macro View

HLD is the general system design — the overall architecture of the application. It describes system architecture, database design, and gives a brief account of the systems, services, and platforms involved, plus the relationships among modules. Also called macro-level or system design.

Think: the architect's floor plan — rooms, walls, and how they connect — not the wiring of each socket.

HLD AT A GLANCE

CREATED BY

The solution architect

INPUT

Software Requirement Spec (SRS)

CONVERTS

Business/client requirement → high-level solution

OUTPUT

Database design, functional design, review record

WHEN

FIRST — before Low Level Design

Low Level Design — The Micro View

LLD is HLD in detail — the component-level design process. It gives a detailed description of every module, the actual logic for each system component, going deep into each module's specification. Also called micro-level or detailed design.

Think: the electrician's wiring diagram for one room — every socket, every wire, every switch.

LLD AT A GLANCE

CREATED BY

Designers and developers

INPUT

Reviewed High Level Design (HLD)

CONVERTS

High-level solution → detailed solution

OUTPUT

Program specification, unit test plan

WHEN

SECOND — after High Level Design

From Requirement to Running Code

Each stage's OUTPUT becomes the next stage's INPUT — and each is verified before it flows on.

SRS
What the client needs

→ *verify*

HLD
Overall architecture

→ *verify*

LLD
Module-level logic

→ *verify*

CODE
The built product

READING THE PIPELINE

The solution architect turns the SRS into HLD. Designers and developers turn reviewed HLD into LLD. Programmers turn LLD into code. Verification sits on every arrow — a fault that slips through one stage is inherited, and amplified, by every stage after it.

High Level vs. Low Level Design

Aspect	High Level Design (HLD)	Low Level Design (LLD)
Scope	Overall system design	Component-level design process
Also called	Macro level / system design	Micro level / detailed design
Describes	Overall architecture of the application	Detailed description of every module
Detail level	Brief functionality of each module	Detailed functional logic of the module
Created by	Solution architect	Designers and developers
Participants	Design, review & client teams	Design, operations & implementers
Sequence	Created FIRST (before LLD)	Created SECOND (after HLD)
Input	Software Requirement Spec (SRS)	Reviewed High Level Design (HLD)
Converts	Client requirement → high-level solution	High-level solution → detailed solution
Output	DB design, functional design, review record	Program specification, unit test plan

What 'Verifying a Design' Actually Means

VERIFYING THE HLD

Does the architecture faithfully cover the SRS?

– Every SRS requirement maps to some module.
– Module relationships and interfaces are consistent.
– Database and platform choices fit the requirements.
– No missing subsystem; no orphan module.

Reviewed by design, review & client teams.

VERIFYING THE LLD

Does each module's logic faithfully realize the HLD?

– Each module's logic matches its HLD responsibility.
– Edge cases, error paths, and inputs are specified.
– The unit test plan actually covers the logic.
– No ambiguity left for the programmer to guess.

Reviewed by design, operations teams & implementers.